Light

Programming Fundamentals

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Data types
2. Variables and constants
3. Operators
4. Selection and iteration
Worked examples
Quick reference
Typical pitfalls

AQA GCSE Computer Science — **Programming Fundamentals (8525 Paper 1)**. This topic covers data types, variables, constants, arithmetic operators, relational operators, Boolean operators, selection statements, iteration, and input/output. These are the building blocks of all programs.

Exam questions ask you to **identify** data types, **write** programs using selection and iteration, **trace** algorithms, **explain** the purpose of code, and **correct** errors. You must use meaningful identifier names and appropriate data types. This is Paper 1 content.

Why programming fundamentals matter

Every program, from simple calculators to complex games, is built from the same fundamental building blocks: storing data in variables, making decisions with selection, and repeating actions with iteration.

Mastering these fundamentals allows you to write programs that solve real problems. In the exam, you'll need to read, write, trace, and debug code using these

concepts. The examples here use Python, but the principles apply to all programming languages.

What you must know

This checklist covers the key exam-visible concepts from the AQA specification. You should be able to explain and use each one.

- **Understand** and use the data types: integer, real, Boolean, character, string.
- **Use** variable declaration, constant declaration, and assignment.
- **Explain** why named constants and variables are used.
- **Use** meaningful identifier names and explain why they are important.
- **Use** arithmetic operators: +, -, *, /, DIV (integer division), MOD (remainder).
- **Use** relational operators: =, ≠, <, >, ≤, ≥.
- **Use** Boolean operators: NOT, AND, OR.
- **Use** selection statements (IF...THEN...ELSE) including nested selection.
- **Use** definite iteration (FOR loops) and indefinite iteration (WHILE, REPEAT...UNTIL).
- **Use** nested iteration structures.
- **Obtain** user input from the keyboard and output data to the display.

> **Specification note.** AQA exams will specify the required format: pseudo-code, program code, or flowchart. When writing program code, use meaningful identifier names, appropriate data types, and clear logic flow. Python uses `int()` for integers and `float()` for real numbers.

Toolkit

Data types at a glance

Data type	Description	Examples	Python
Integer	Whole number (positive, negative, or zero)	-5, 0, 42, 1000	`int`
Real	Number with a decimal point	3.14, -0.5, 99.99	`float`
Boolean	True or False value	True, False	`bool`
Character	A single letter, digit, or symbol	'A', '7', '@'	`str` (length 1)
String	A sequence of characters (text)	"Hello", "GCSE"	`str`

Operators reference

Arithmetic

- + Addition
- - Subtraction

Relational

- = or == Equal to
- ≠ or != Not equal to

- `*` Multiplication
- `/` Real division
- `DIV` or `//` Integer division
- `MOD` or `%` Remainder
- `<` Less than
- `>` Greater than
- `≤` or `<=` Less than or equal
- `≥` or `>=` Greater than or equal

Boolean

- `NOT` or `not` — Reverses value
- `AND` or `and` — Both must be True
- `OR` or `or` — At least one True

Selection and iteration structures

Structure	AQA Pseudo-code	Python
Selection	`IF...THEN...ELSE...ENDIF`	`if...elif...else:`
Definite iteration	`FOR i ← 1 TO n...ENDFOR`	`for i in range(n):`
Indefinite (condition at start)	`WHILE condition...ENDWHILE`	`while condition:`
Indefinite (condition at end)	`REPEAT...UNTIL condition`	No direct equivalent (use `while True` with `break`)

1. Data types

A **data type** defines what kind of data a variable can hold and what operations can be performed on it. Choosing the correct data type is essential for programs to work correctly.

1.1 Integer

An **integer** is a whole number — positive, negative, or zero. Integers have no decimal point.

```
AQA Pseudo-code:
age ← 16
score ← -5
count ← 0
```

```
# Python:
age = 16
score = -5
count = 0
```

Use integers for: counting, ages, scores, quantities, positions in a list.

1.2 Real (float)

A **real** number (called `float` in Python) has a decimal point. Use reals when you need precision or fractional values.

```
AQA Pseudo-code:
price ← 19.99
temperature ← -3.5
pi ← 3.14159
```

```python
# Python:
price = 19.99
temperature = -3.5
pi = 3.14159
```

Use reals for: prices, measurements, percentages, scientific calculations.

1.3 Boolean

A **Boolean** can only have two values: `True` or `False`. Booleans are used for conditions and flags.

```
AQA Pseudo-code:
gameOver ← False
loggedIn ← True
isValid ← x > 10
```

```python
# Python:
game_over = False
logged_in = True
is_valid = x > 10
```

Use Booleans for: conditions in IF statements, loop control, flags to track state.

1.4 Character

A **character** is a single letter, digit, or symbol. In Python, characters are just strings of length 1.

```
AQA Pseudo-code:
grade ← 'A'
digit ← '7'
symbol ← '@'
```

```python
# Python:
grade = 'A'
```

```
digit = '7'
symbol = '@'
```

Use characters for: single letters, grades, menu options, individual symbols.

1.5 String

A **string** is a sequence of characters — text data. Strings are enclosed in quotes.

```
AQA Pseudo-code:
name ← "Alice"
message ← "Hello, World!"
empty ← ""
```

```
# Python:
name = "Alice"
message = "Hello, World!"
empty = ""
```

Use strings for: names, addresses, messages, any text data.

Worked Example 1 – Identifying data types

Question: State the most appropriate data type for each of the following: (a) A student's name (b) The number of students in a class (c) Whether a student has paid their fees (d) A student's average mark. [4 marks]

Model answer:
(a) String **(1)**
(b) Integer **(1)**
(c) Boolean **(1)**
(d) Real **(1)**

Exam tip: Think about the values: whole numbers = integer, decimals = real, yes/no = Boolean, text = string.

2. Variables and constants

Programs store data in **variables** and **constants**. Understanding when to use each is important for writing good code.

2.1 Variables

A **variable** is a named storage location that can hold a value. The value can change during program execution.

Variable: A named location in memory that stores a value which can change while the program is running.

```
AQA Pseudo-code:
score ← 0
score ← score + 10
name ← USERINPUT
```

```python
# Python:
score = 0
score = score + 10
name = input()
```

2.2 Constants

A **constant** is a named value that does not change during program execution. Constants make programs easier to understand and maintain.

Constant: A named value that is set once and does not change while the program is running.

```
AQA Pseudo-code:
CONST VAT_RATE ← 0.20
CONST MAX_PLAYERS ← 4
CONST PI ← 3.14159
```

```python
# Python (convention: uppercase names):
VAT_RATE = 0.20
MAX_PLAYERS = 4
PI = 3.14159
```

2.3 Why use constants?

- **Easier to understand:** `price * VAT_RATE` is clearer than `price * 0.20`
- **Easier to maintain:** If VAT changes, you only update one line
- **Prevents accidental changes:** Constants cannot be modified during execution
- **Reduces errors:** Using a named constant avoids typing the wrong value

2.4 Meaningful identifier names

An **identifier** is the name given to a variable, constant, or subroutine. Meaningful names make code easier to read and understand.

Good identifier names

- `totalScore`
- `studentName`
- `isGameOver`
- `numberOfAttempts`
- `VAT_RATE`

Poor identifier names

- `x, y, z`
- `temp, data`
- `a1, thing`
- `n, s`
- `var1, var2`

Why meaningful names matter: In exams, you are expected to use meaningful identifier names. Poor names can lose marks. Names should describe what the variable stores or what the subroutine does.

2.5 Assignment

Assignment stores a value in a variable. In AQA pseudo-code, the assignment operator is ←. In Python, it is =.

```
AQA Pseudo-code:
x ← 10            # Assign 10 to x
x ← x + 1         # Increment x by 1
y ← x * 2         # Assign x*2 to y
```

```
# Python:
x = 10            # Assign 10 to x
x = x + 1         # Increment x by 1
y = x * 2         # Assign x*2 to y
```

Worked Example 2 — Variables vs constants

Question: A program calculates the cost of items including VAT at 20%. Explain why VAT_RATE should be stored as a constant rather than a variable. [2 marks]

Model answer:
The VAT rate should not change during program execution **(1)**. Using a constant prevents accidental modification and makes the code easier to maintain — if the rate changes, only one line needs updating **(1)**.

Exam tip: Constants are for values that shouldn't change. Key benefits: prevents accidental changes, easier maintenance, clearer code.

Worked Example 3 — Meaningful identifier names

Question: Explain why it is important to use meaningful identifier names in programs. [2 marks]

Model answer:
Meaningful names make the code easier to read and understand **(1)**. This helps other programmers (or yourself later) to maintain, debug, and modify the code **(1)**.

Exam tip: Link to readability AND maintenance. Both points are needed for full marks.

3. Operators

Operators perform actions on values. AQA requires you to understand three types: arithmetic, relational, and Boolean operators.

3.1 Arithmetic operators

Arithmetic operators perform mathematical calculations.

Operator	Name	Pseudo-code	Python	Example	Result
+	Addition	`5 + 3`	`5 + 3`	5 + 3	8
-	Subtraction	`10 - 4`	`10 - 4`	10 - 4	6
*	Multiplication	`6 * 7`	`6 * 7`	6 × 7	42
/	Real division	`7 / 2`	`7 / 2`	7 ÷ 2	3.5
DIV	Integer division	`11 DIV 2`	`11 // 2`	11 ÷ 2 (whole part)	5
MOD	Remainder	`11 MOD 2`	`11 % 2`	Remainder of 11 ÷ 2	1

DIV and MOD explained

Integer division (DIV) and remainder (MOD) work together:

- **11 DIV 2 = 5** — How many times does 2 go into 11? (5 times)
- **11 MOD 2 = 1** — What is left over? (1)

Together: 11 = (5 × 2) + 1

Common uses of MOD:

- Check if a number is even: `n MOD 2 = 0`
- Get the last digit: `n MOD 10`
- Wrap around (e.g., clock): `hour MOD 12`

3.2 Relational operators

Relational operators compare two values and return a Boolean result (True or False).

Operator	Meaning	Pseudo-code	Python	Example	Result
=	Equal to	`x = 5`	`x == 5`	5 = 5	True
≠	Not equal to	`x ≠ 5`	`x != 5`	3 ≠ 5	True
<	Less than	`x < 10`	`x < 10`	3 < 10	True
>	Greater than	`x > 10`	`x > 10`	3 > 10	False
≤	Less than or equal	`x ≤ 10`	`x <= 10`	10 ≤ 10	True
≥	Greater than or equal	`x ≥ 10`	`x >= 10`	10 ≥ 10	True

3.3 Boolean operators

Boolean operators combine or modify Boolean values. They are used to create complex conditions.

Operator	Meaning	Result
NOT	Reverses the value	NOT True = False; NOT False = True
AND	Both must be True	True AND True = True; all others = False
OR	At least one must be True	False OR False = False; all others = True

Boolean truth tables

NOT

A	NOT A
True	False
False	True

AND

A	B	A AND B
True	True	True
True	False	False
False	True	False
False	False	False

OR

A	B	A OR B
True	True	True
True	False	True
False	True	True

False	False	False

```python
# Example: Combining Boolean operators in Python
age = 17
has_ticket = True

# Can enter if 18+ OR has a ticket
if age >= 18 or has_ticket:
    print("Entry allowed")

# Must be 18+ AND have a ticket
if age >= 18 and has_ticket:
    print("VIP entry")
```

Worked Example 4 — DIV and MOD

Question: State the values of the following expressions:
(a) 17 DIV 5 (b) 17 MOD 5 (c) 24 MOD 6 (d) 100 DIV 7 [4 marks]

Model answer:
(a) 17 DIV 5 = **3** *(17 ÷ 5 = 3 remainder 2, integer part is 3)*
(b) 17 MOD 5 = **2** *(remainder when 17 is divided by 5)*
(c) 24 MOD 6 = **0** *(24 ÷ 6 = 4 exactly, no remainder)*
(d) 100 DIV 7 = **14** *(100 ÷ 7 = 14 remainder 2, integer part is 14)*

Exam tip: DIV gives the whole number part. MOD gives the remainder. Practice these — they appear frequently!

Worked Example 5 — Boolean expressions

Question: Given x = 5 and y = 10, state whether each expression is True or False:
(a) x < y AND x > 0 (b) x = 10 OR y = 10 (c) NOT (x = y) [3 marks]

Model answer:
(a) x < y AND x > 0 → 5 < 10 AND 5 > 0 → True AND True = **True**
(b) x = 10 OR y = 10 → False OR True = **True**
(c) NOT (x = y) → NOT (5 = 10) → NOT False = **True**

Exam tip: Work step by step. Evaluate each part of the expression, then combine using the Boolean operator.

4. Selection and iteration

Programs need to make decisions (**selection**) and repeat actions (**iteration**). These control structures, along with **sequence**, are the three building blocks of all programs.

4.1 Selection (IF statements)

Selection allows a program to choose which code to execute based on a condition.

```
Simple IF:
IF score >= 50 THEN
    OUTPUT "Pass"
ENDIF

IF...ELSE:
IF score >= 50 THEN
    OUTPUT "Pass"
ELSE
    OUTPUT "Fail"
ENDIF

IF...ELSE IF...ELSE:
IF score >= 70 THEN
    OUTPUT "Distinction"
ELSE IF score >= 50 THEN
    OUTPUT "Pass"
ELSE
    OUTPUT "Fail"
ENDIF
```

```python
# Simple IF:
if score >= 50:
    print("Pass")

# IF...ELSE:
if score >= 50:
    print("Pass")
else:
    print("Fail")

# IF...ELIF...ELSE:
if score >= 70:
    print("Distinction")
elif score >= 50:
    print("Pass")
else:
    print("Fail")
```

4.2 Nested selection

Nested selection is when one IF statement is placed inside another. This allows for more complex decision-making.

```
IF gameWon THEN
    OUTPUT "You won!"
    IF score > highScore THEN
        OUTPUT "New high score!"
```

```
        highScore ← score
    ENDIF
ELSE
    OUTPUT "Try again"
ENDIF
```

```python
if game_won:
    print("You won!")
    if score > high_score:
        print("New high score!")
        high_score = score
else:
    print("Try again")
```

4.3 Definite iteration (FOR loops)

Definite iteration (count-controlled) repeats a block of code a specific number of times.

```
# Repeat 5 times:
FOR i ← 1 TO 5
    OUTPUT i
ENDFOR

# Output: 1, 2, 3, 4, 5
```

```python
# Repeat 5 times:
for i in range(1, 6):
    print(i)

# Output: 1, 2, 3, 4, 5
```

> **Python range() note:** `range(1, 6)` gives 1, 2, 3, 4, 5 (stops BEFORE 6). In AQA pseudo-code, `FOR i ← 1 TO 5` includes both 1 and 5.

4.4 Indefinite iteration (WHILE and REPEAT)

Indefinite iteration (condition-controlled) repeats until a condition is met. The number of repetitions is not known in advance.

Condition at the start (WHILE)

The condition is checked **before** each iteration. The loop may execute zero times if the condition is initially False.

```
total ← 0
number ← USERINPUT
WHILE number ≠ -1
    total ← total + number
    number ← USERINPUT
```

```
ENDWHILE
OUTPUT total
```

```
total = 0
number = int(input())
while number != -1:
    total = total + number
    number = int(input())
print(total)
```

Condition at the end (REPEAT...UNTIL)

The condition is checked **after** each iteration. The loop always executes at least once.

```
REPEAT
    password ← USERINPUT
UNTIL password = "secret"
OUTPUT "Access granted"
```

```
# Python equivalent (no REPEAT...UNTIL)
while True:
    password = input()
    if password == "secret":
        break
print("Access granted")
```

4.5 Nested iteration

Nested iteration is when one loop is placed inside another. This is commonly used for working with 2D structures or generating combinations.

```
# Print a 3x3 grid of stars
FOR row ← 1 TO 3
    FOR col ← 1 TO 3
        OUTPUT "*"
    ENDFOR
    OUTPUT newline
ENDFOR
```

```
# Print a 3x3 grid of stars
for row in range(3):
    for col in range(3):
        print("*", end="")
    print()  # New line
```

Output:

```
***
***
***
```

4.6 Input and output

Programs need to receive data from users (**input**) and display results (**output**).

```
# Input:
name ← USERINPUT
age ← USERINPUT

# Output:
OUTPUT "Hello, " + name
OUTPUT "You are " + age
```

```python
# Input:
name = input("Enter name: ")
age = int(input("Enter age: "))

# Output:
print("Hello,", name)
print("You are", age)
```

Python input() note: The `input()` function always returns a string. To get a number, you must convert it: `int(input())` for integers, `float(input())` for reals.

Worked Example 6 — Writing a FOR loop

Question: Write a Python program that outputs the numbers 1 to 10. [2 marks]

Model answer:

```python
for i in range(1, 11):
    print(i)
```

Exam tip: Remember `range(1, 11)` stops before 11, so it gives 1 to 10.

Worked Example 7 — Writing a WHILE loop

Question: Write a pseudo-code algorithm that keeps asking the user to enter a number until they enter 0, then outputs the total of all numbers entered. [4 marks]

Model answer:

```
total ← 0
number ← USERINPUT
WHILE number ≠ 0
    total ← total + number
    number ← USERINPUT
ENDWHILE
OUTPUT total
```

Exam tip: Initialise the total to 0 before the loop. Get input before AND inside the loop for WHILE loops.

Worked Example 8 — Nested selection

Question: Write a Python program that asks for a user's age and whether they have a ticket. They can enter if they are 18 or over, OR if they have a ticket (regardless of age). [4 marks]

Model answer:

```python
age = int(input("Enter your age: "))
has_ticket = input("Do you have a ticket? (yes/no): ")

if age >= 18 or has_ticket == "yes":
    print("Entry allowed")
else:
    print("Entry denied")
```

Exam tip: Use OR when either condition allows entry. Use AND when both conditions are required.

Worked Example 9 — Nested iteration

Question: Write a Python program that prints a multiplication table for numbers 1 to 5 (i.e., 1×1 up to 5×5). [4 marks]

Model answer:

```python
for i in range(1, 6):
    for j in range(1, 6):
```

```
        print(i * j, end=" ")
    print()
```

Exam tip: The outer loop controls rows; the inner loop controls columns. Each outer iteration completes one full row.

Worked Example 10 — WHILE vs FOR

Question: Explain when you would use a WHILE loop rather than a FOR loop. [2 marks]

Model answer:
Use a WHILE loop when you don't know in advance how many times the loop will repeat **(1)**, for example, when waiting for user input or processing data until a condition is met **(1)**.

Exam tip: FOR = known number of iterations. WHILE = unknown number of iterations.

Worked Example 11 — Input validation loop

Question: Write a Python program that asks the user to enter a number between 1 and 10, and keeps asking until they enter a valid number. [4 marks]

Model answer:

```
number = int(input("Enter a number between 1 and 10: "))
while number < 1 or number > 10:
    print("Invalid. Try again.")
    number = int(input("Enter a number between 1 and 10: "))
print("You entered:", number)
```

Exam tip: Input validation is a common exam question. The loop continues WHILE the input is invalid.

Worked Example 12 — Complete program

Question: Write a Python program that calculates the average of 5 numbers entered by the user. [5 marks]

Model answer:

```
total = 0
for i in range(5):
    number = float(input("Enter a number: "))
    total = total + number
average = total / 5
print("The average is:", average)
```

Exam tip: Key steps: initialise total to 0, use a loop to input 5 numbers, add each to total, divide by 5. Use float() since average may have decimals.

Extended response practice. AQA exams include extended response questions. Here is an example combining multiple concepts.

Extended Response — Program design

Question: A shop wants a program to calculate the total cost of items. The user enters prices until they enter 0. If the total is over £50, a 10% discount is applied. The program should display the final total.
Write a Python program to solve this problem. [6 marks]

Model answer (Level 3, 5—6 marks):

```
DISCOUNT_THRESHOLD = 50
DISCOUNT_RATE = 0.10

total = 0
price = float(input("Enter price (0 to finish): "))

while price != 0:
    total = total + price
    price = float(input("Enter price (0 to finish): "))

if total > DISCOUNT_THRESHOLD:
    discount = total * DISCOUNT_RATE
    total = total - discount
    print("Discount applied!")

print("Total: £", total)
```

Exam tip (6-mark questions): For full marks: use meaningful variable names, use constants for fixed values, handle input correctly, include the loop and

selection logic, and produce correct output. Test your logic mentally with example values.

Worked examples summary

This page contains 12 worked examples plus one extended response, covering all the key programming fundamentals.

Example	Topic	Question type	Marks
1	Identifying data types	State	4
2	Variables vs constants	Explain	2
3	Meaningful identifier names	Explain	2
4	DIV and MOD	Calculate	4
5	Boolean expressions	Evaluate	3
6	Writing a FOR loop	Write code	2
7	Writing a WHILE loop	Write pseudo-code	4
8	Nested selection with OR	Write code	4
9	Nested iteration	Write code	4
10	WHILE vs FOR	Explain	2
11	Input validation loop	Write code	4
12	Complete program (average)	Write code	5
Ext.	Program design (shop discount)	Write code (extended)	6

Quick reference

Data types summary

Data type	Use for	Python
Integer	Counting, ages, whole numbers	`int`
Real	Prices, measurements, averages	`float`
Boolean	Conditions, flags, yes/no	`bool`
Character	Single letters, grades	`str`
String	Names, text, messages	`str`

Python syntax cheat sheet

Input/Output

```
name = input("Prompt: ")
age = int(input("Age: "))
print("Hello", name)
```

Operators

```
# Integer division and remainder
x // y  # DIV
x % y   # MOD
```

Selection

```
if condition:
    # code
elif other_condition:
    # code
else:
    # code
```

Iteration

```
for i in range(5):
    # repeats 5 times

while condition:
    # repeats while True
```

Typical pitfalls

Pitfall 1 — Wrong data type.
Wrong: Using string for a number you need to calculate with.
Right: Use `int(input())` or `float(input())` for numbers.

Pitfall 2 — Confusing = and ==.
Wrong: `if x = 5:` (assignment)
Right: `if x == 5:` (comparison)

Pitfall 3 — Off-by-one errors in range().
Wrong: `range(1, 10)` for 1 to 10
Right: `range(1, 11)` — range stops BEFORE the second number.

Pitfall 4 — Forgetting to initialise variables.
Wrong: Using `total` before setting it to 0.
Right: Always initialise: `total = 0` before a loop.

Pitfall 5 — Wrong operator for conditions.
Wrong: `x < 10 AND x < 1` (both can't be true)
Right: Think carefully: use OR when either is acceptable, AND when both are required.

Pitfall 6 — Poor identifier names.
Wrong: `x`, `a`, `temp`
Right: `totalScore`, `studentName`, `isValid`

Pitfall 7 — Infinite loops.
Wrong: Condition never becomes False.
Right: Ensure something inside the loop changes the condition.

Pitfall 8 — Confusing DIV and MOD.
Wrong: `17 MOD 5 = 3`
Right: DIV = 3 (how many times), MOD = 2 (remainder).

Pitfall 9 — Missing indentation.
Wrong: Code not indented inside if/for/while.
Right: Python requires correct indentation for blocks.

Pitfall 10 — Not testing boundary conditions.
Wrong: `if age > 18` when 18 should be included.
Right: `if age >= 18` — check the boundary carefully.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.